40 POSTGRES END TO END — FROM ZERO SERVER TO A WORKING NESTJS CRUD MODULE

theory/36 is a side-by-side CRUD comparison against the Mongo module you already have. theory/37 is a pure SQL reference. This file is different: it's the single, start-to-finish walkthrough — assuming NOTHING exists yet — of getting a real Postgres server running and a full NestJS module talking to it, in order, with no Mongo detours.

STEP 1 — GET A POSTGRES SERVER RUNNING

Pick ONE of these (you only need a server you can reach by URI):

a) Docker (fastest, no local install, easiest to throw away): docker run --name pg-dev -e POSTGRES_PASSWORD=devpass -p 5432:5432 -d postgres Check it's up: docker ps

b) Local install (postgresql.org installer for Windows): installs a Postgres service + pgAdmin (GUI) + psql (CLI), running on localhost:5432 with whatever password you set during install.

c) Hosted/free tier (Neon, Supabase, Railway) — no install at all, you just get a connection URI from their dashboard. Good for "want to try this without touching my machine."

Whichever you pick, you end up with a connection string shaped like:

```
postgres://<user>:<password>@<host>:<port>/<database>
postgres://postgres:devpass@localhost:5432/nest_crud
```

STEP 2 — CREATE THE DATABASE ITSELF

A fresh Postgres server has no `nest_crud` database yet — the server and the database are two different things (one server can host many databases).

Via psql (CLI):

```
psql -U postgres -h localhost
CREATE DATABASE nest_crud;
\q
```

Via Docker, same idea, run psql INSIDE the container:

```
docker exec -it pg-dev psql -U postgres -c "CREATE DATABASE nest_crud;"
```

Via pgAdmin: right-click "Databases" → Create → Database → name it nest_crud.

Once this exists, your full URI is:

```
postgres://postgres:devpass@localhost:5432/nest_crud
```

STEP 3 — INSTALL THE NESTJS PACKAGES

```
npm install @nestjs/typeorm typeorm pg
```

@nestjs/typeorm	→ Nest's wrapper module (TypeOrmModule), same role @nestjs/mongoose plays for Mongo
typeorm	→ the actual ORM — entities, repositories, query builder
pg	→ the low-level Postgres driver TypeORM talks through (same role the mongodb driver plays under Mongoose)

STEP 4 — ENV VARS (same pattern as MONGODB_URI, theory/28/29)

Add to .env:

```
POSTGRES_URI=postgres://postgres:devpass@localhost:5432/nest_crud
```

Never commit real credentials — this follows the same ConfigModule/.env rules as theory/28-environment-variables.txt.

STEP 5 — WIRE UP THE CONNECTION IN app.module.ts

```
import { TypeOrmModule } from '@nestjs/typeorm';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    TypeOrmModule.forRootAsync({
      inject: [ConfigService],
      useFactory: (configService: ConfigService) => ({
        type: 'postgres',
        url: configService.getOrThrow<string>('POSTGRES_URI'),
        autoLoadEntities: true, // auto-registers every entity passed to any forFeature() call, so you don't list them all here by hand
        synchronize: true,      // DEV ONLY (see STEP 10 — never in production)
      }),
    }),
    StudentPgModule,
  ],
})
export class AppModule {}
```

This can sit right alongside an existing MongooseModule.forRootAsync() import — a Nest app can hold two completely separate DB connections at once, nothing about adding Postgres requires removing Mongo.

STEP 6 — DEFINE THE ENTITY (TypeORM's version of a Mongoose @Schema)

```
// student-pg/student.entity.ts
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn, UpdateDateColumn } from 'typeorm';
```

```ts
@Entity('students')
export class Student {
  @PrimaryGeneratedColumn()
  id!: number;               // Postgres needs an explicit PK column — SERIAL under the hood (theory/37 section 1)

  @Column()
  name!: string;

  @Column()
  age!: number;

  @Column({ unique: true })
  email!: string;

  @Column({ nullable: true })  // nullable -> optional column, like course?: string in a DTO
  course?: string;

  @CreateDateColumn()
  createdAt!: Date;

  @UpdateDateColumn()
  updatedAt!: Date;
}
```

Every @Column() maps straight onto a real SQL column type and constraint — `unique: true` here IS `UNIQUE` from theory/37 section 10, enforced by Postgres itself, not just app-level validation.

STEP 7 — DTOs (identical to the Mongo ones — class-validator doesn't care which DB it feeds)

```ts
// student-pg/dto/create-student.dto.ts
export class CreateStudentDto {
  @IsString() name!: string;
  @IsInt() age!: number;
  @IsEmail() email!: string;
  @IsOptional() @IsString() course?: string;
}
```

Reuses the exact same class-validator/class-transformer pattern from theory/13-dto.txt and theory/15-class-validator-and-class-transformer.txt.

STEP 8 — MODULE: REGISTER THE ENTITY (mirrors MongooseModule.forFeature)

```ts
// student-pg/student-pg.module.ts
@Module({
  imports: [TypeOrmModule.forFeature([Student])],
  controllers: [StudentPgController],
  providers: [StudentPgService],
})
export class StudentPgModule {}
```

forFeature([Student]) is what makes @InjectRepository(Student) below resolvable — same role theory/32's forFeature() plays for @InjectModel.

STEP 9 — SERVICE: FULL CRUD WITH Repository<Student>

```
// student-pg/student-pg.service.ts
@Injectable()
export class StudentPgService {
  constructor(
    @InjectRepository(Student) private studentRepo: Repository<Student>,
  ) {}

  async create(dto: CreateStudentDto): Promise<Student> {
    const student = this.studentRepo.create(dto); // builds an entity instance in memory, not saved yet
    return this.studentRepo.save(student);          // INSERT ... RETURNING * under the hood
  }

  async findAll(): Promise<Student[]> {
    return this.studentRepo.find();                  // SELECT * FROM students
  }

  async findOne(id: number): Promise<Student> {
    const student = await this.studentRepo.findOneBy({ id }); // SELECT * FROM students WHERE id = $1
    if (!student) {
      throw new NotFoundException(`Student ${id} not found`);
    }
    return student;
  }

  async update(id: number, dto: UpdateStudentDto): Promise<Student> {
    const student = await this.studentRepo.preload({ id, ...dto }); // loads the row, merges dto's
fields on top — like findByIdAndUpdate's merge behavior
    if (!student) {
      throw new NotFoundException(`Student ${id} not found`);
    }
    return this.studentRepo.save(student); // UPDATE ... WHERE id = $1
  }

  async remove(id: number): Promise<Student> {
    const student = await this.findOne(id); // reuse — also gives us the row to return after deleting it
    await this.studentRepo.delete(id);        // DELETE FROM students WHERE id = $1
    return student;
  }
}
```

Every method is still async/Promise-based for the exact reason theory/32 explains for Mongoose — this is a network round-trip, not an in-memory array read, regardless of which DB is on the other end.

STEP 10 — CONTROLLER (same routes/verbs/shape as student.controller.ts)

```
@Controller('student-pg')
export class StudentPgController {
  constructor(private readonly studentPgService: StudentPgService) {}

  @Post('createstudent')
```

```
  create(@Body() dto: CreateStudentDto) {
    return this.studentPgService.create(dto);
  }

  @Get('getallstudents')
  findAll() {
    return this.studentPgService.findAll();
  }

  @Get('getstudent/:id')
  findOne(@Param('id', ParseIntPipe) id: number) { // ParseIntPipe: Postgres ids are numbers, not Mongo ObjectId strings — theory/09/10 cover pipes
    return this.studentPgService.findOne(id);
  }

  @Patch('updatestudent/:id')
  update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateStudentDto) {
    return this.studentPgService.update(id, dto);
  }

  @Delete('deletestudent/:id')
  remove(@Param('id', ParseIntPipe) id: number) {
    return this.studentPgService.remove(id);
  }
}
```

ParseIntPipe is the one real wiring difference from the Mongo controller — Mongo ids are 24-char hex strings taken as-is, Postgres ids are integers, so the route param needs converting before it hits a `WHERE id = $1` comparison.

STEP 11 — RELATIONS (the Postgres-native equivalent of theory/38's referencing)

```
@Entity('courses')
export class Course {
  @PrimaryGeneratedColumn() id!: number;
  @Column() title!: string;

  @OneToMany(() => Student, (student) => student.course)
  students!: Student[];
}

@Entity('students')
export class Student {
  ...
  @ManyToOne(() => Course, (course) => course.students)
  course!: Course;       // TypeORM auto-creates a courseId FK column for you
}
```

Fetching WITH the relation loaded (TypeORM's populate() equivalent):

```
this.studentRepo.find({ relations: ['course'] });
// -> SELECT ... then a JOIN (or a second query, depending on strategy)
// behind the scenes — same two-query-vs-one-join tradeoff theory/38
// calls out for Mongoose's populate()
```

Unlike Mongo's `ref`, this ACTUALLY creates a foreign key constraint in Postgres — inserting a Student with a non-existent course id is rejected by the database itself (theory/37 section 10 / theory/38 section 6, the exact gap Mongo references don't close).

STEP 12 — synchronize: true vs MIGRATIONS

`synchronize: true` (used above) makes TypeORM inspect your @Entity classes on every boot and auto-create/alter tables to match — zero manual CREATE TABLE needed in dev. This is DEV-ONLY: in production it can silently drop or alter real columns/data if an entity changes.

Production workflow instead:

```
    npx typeorm migration:generate -n AddStudents   // diffs entities vs current DB, writes a migration file
    npx typeorm migration:run                        // applies pending migrations
This is the deploy-safe version of the ALTER TABLE statements in
theory/37 section 1 — tracked, reviewable, and run deliberately instead
of automatically on every app boot.
```

STEP 13 — RUN IT AND VERIFY

```
npm run start:dev
```

With synchronize: true, the `students` table appears automatically on first boot — confirm with:

```
docker exec -it pg-dev psql -U postgres -d nest_crud -c "\dt"
docker exec -it pg-dev psql -U postgres -d nest_crud -c "SELECT * FROM students;"
```

Then hit the routes (same curl examples as theory/36):

```
curl -X POST http://localhost:3000/student-pg/createstudent \
  -H "Content-Type: application/json" \
  -d "{\"name\":\"Rahul\",\"age\":20,\"email\":\"rahul@example.com\",\"course\":\"nid\"}"

curl http://localhost:3000/student-pg/getallstudents
curl http://localhost:3000/student-pg/getstudent/1
curl -X PATCH http://localhost:3000/student-pg/updatestudent/1 -H "Content-Type: application/json" -d "{\"age\":21}"
curl -X DELETE http://localhost:3000/student-pg/deletestudent/1
```

★ RULE OF THUMB

Server → database → packages → env var → TypeOrmModule connection → @Entity → DTOs → forFeature() registration → Repository-based service → controller. That's the whole path, every step mirroring a piece you already built for Mongo — the only genuinely new concepts are the hard schema (columns + constraints enforced by the DB) and FOREIGN KEY relations actually being checked, instead of just suggested.

See also: theory/36-postgres-setup-and-crud.txt (Mongo-vs-Postgres comparison), theory/37-sql-cheatsheet.txt (raw SQL reference), theory/38-mongodb-relationships.txt and theory/39-mongodb-relationships-simple.txt (the Mongo

side of STEP 11), **theory/28-environment-variables.txt**, **theory/29-nestjs-config-module.txt**, **theory/13-dto.txt**